

Advanced Natural Language Processing: AdvNLP

*Lesson 1: Introduction, tokenization, and
early language & embedding models*

Table of Contents

1. Introduction and organization
2. Tokenization methods
3. Conditional text generation & statistical language models
4. Non-contextual word embeddings with neural networks

Appendix: LM evaluation metrics and statistical LMs

1. ORGANIZATION AND PLAN OF THE COURSE

Teaching details

- Format:
 - 100% on-site with in-person attendance
 - on demand, passive streaming and recording on Zoom (group to be created)
- Resources on Moodle: <https://moodle.msengineering.ch/course/view.php?id=3134>
- Schedule: 14 weeks
 - 2-period lecture (13:10-13:55 + 14:05-14:50)
 - 2-period practical work (15:00-15:45 + 15:55-16:40)
 - some labs will be interspersed with lectures
- Course grades
 - exam (two hours): 80% | 4-5 graded labs: 20%
 - Python/Jupyter, laptop/Colab, 2 people

MSE Teachers of TSM AdvNLP: Lausanne and Zurich



Fabrizio Albertetti

HES-SO/HE-Arc, Saint-Imier (BE)
PhD University of Neuchâtel, Switzerland
<https://people.he-arc.ch/fabrizio.albertetti>



Andrei Popescu-Belis (*head of module*)

HES-SO/HEIG-VD, Yverdon-les-Bains (VD)
PhD University of Paris-Sud, France
<http://iict-space.heig-vd.ch/apu/>



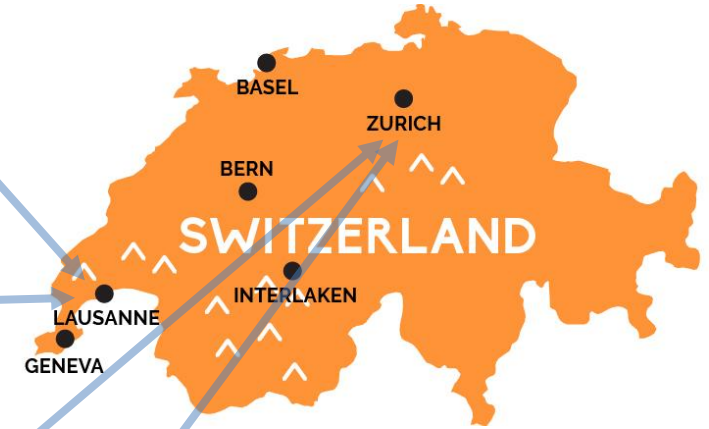
Daniele Puccinelli

SUPSI, Manno (TI)
PhD University of Notre-Dame, USA
<http://people.dti.supsi.ch/~puccinelli/>



Daniel Perruchoud

FHNW, Windisch (AG)
PhD ETHZ, Switzerland
<https://www.fhnw.ch/en/people/daniel-perruchoud>



Plan of the course

Week	Date	Professors	Title
1	19.2.	Daniel Perruchoud	Introduction, tokenization & early language & embedding models
2	26.2.	Daniel Perruchoud	Machine Translation with RNNs & Transformers
3,4	5./12.3.	Daniele Puccinelli	Training Transformer Encoders for non-generative tasks
5	19.3.	Daniele Puccinelli	POS tagging & parsing, with & without Transformer Encoders
6	26.3.	Daniel Perruchoud	Training Transformer Decoders & LLMs for text generation
7	2.4.	Daniele Puccinelli	LLMs & Reinforcement Learning from Human Feedback
8	9.4.	Daniele Puccinelli	LLM evaluation benchmarks & LLM limitations
9	16.4.	Daniel Perruchoud	LLM in-context learning & prompt engineering
10	23.4.	Daniele Puccinelli	LLM implementation – Context size & memory use
11	30.4.	Daniel Perruchoud	Retrieval Augmented Generation
12	7.5.	Daniel Perruchoud	Agentic AI
13	21.5.	Daniel Perruchoud	Speech representation & speech-to-text transcription
14	28.5.	Daniele Puccinelli	Speech synthesis & translation

Textbook

Dan Jurafsky and James H. Martin

Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition

⇒ 2nd edition (printed), Prentice-Hall, 2008

⇒ 3rd edition (online draft), <https://web.stanford.edu/~jurafsky/slp3/>
regularly updated as the domain evolves very quickly!

2. TOKENIZATION METHODS

(Unknown) words in neural networks

- Input a word into a neural network = activate a specific input unit = one-hot vector
 - therefore, texts need to be fragmented into words before processing by a neural network
- The input layer cannot be extended excessively: $10^4 - 10^6$ words from training data
- Therefore, many unknown words appear during testing (out of vocabulary, OOV)
 - although the number of lemmas in a language is around 10^5 , the number of forms (conjugated verbs, plurals, etc.) is $10 - 100 \times$ larger*
 - Also, numbers, proper names (people, places, companies), etc.
 - novel words created by borrowing, compounding**, derivation
 - people make spelling mistakes or use straaaange spellings on purpose
- Possible but non-optimal solutions
 - ignore unknown words; check spelling; normalize words; model numbers with rules; decompose words into root and affixes; use character-based models (unknown words $\rightarrow$ characters)

* This depends on the language :
e.g. EN < FR < RU

** E.g., Donaudampfschifffahrts-
gesellschaftskapitän

Word- / character- /subword-based tokenization

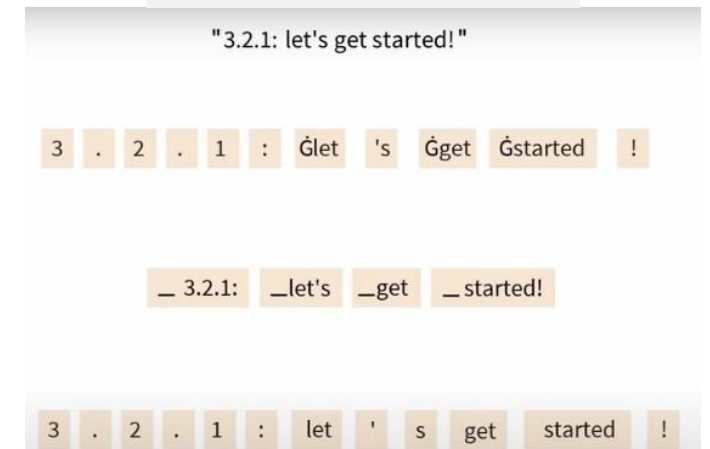
- Word-based: split text into individual words
 - simple and intuitive: use white spaces, deal with punctuation
 - limitations: need to handle many exceptions; resulting vocabulary size $\uparrow\uparrow$; OOVs; no modeling of relationships between words ('dog' $\neq$ 'dogs')
- Character-based: split text into individual characters
 - very simple: no need for specific rules
 - advantage: can handle any word, no OOVs (but possibly unseen characters)
 - limitations: long sequences, much more difficult to learn, hard to capture semantic information
- Subword-based:
 - vocabulary made of words, word parts, and characters, learned from the training data!

```
[ 'There', 'are', 'eight', 'orbit', '##ers', 'surveying', 'the', 'planet', ':', 'Mars', 'Odyssey', ',', 'Mars', 'Express', ',', 'Mars', 'Reconnaissance', 'Or', '##bit', '##er', ',', 'Mars', 'Or', '##bit', '##er', 'Mission', ',', 'MA', '##VE', '##N', ',', 'the', 'Trace', 'Gas', 'Or', '##bit', '##er', ',', 'the', 'T', '##ian', '##wen', '-', '1', 'orbit', '##er', ',', 'and', 'the', 'Hope', 'Mars', 'Mission' ]
```

Byte-Pair Encoding (BPE): preprocessing

- Byte-Pair Encoding requires **input text must be preprocessed** for
 - training = build the vocabulary from the training texts
 - testing = tokenize a new text using this vocabulary
- **Preprocessing** includes
 - clean-up newlines and multiple whitespaces, possibly lowercase
 - tokenize into **words** and **punctuations** based on whitespaces and/or **model-specific rules**
 - mark word endings with a specific symbol, e.g., </w>, or word continuations, e.g., ## or __ (**model-specific rules**)
 - split text into individual characters, remove infrequent ones

Three different
preprocessing
model-specific rules



Source: <https://huggingface.co/learn/nlp-course/chapter6/4?fw=pt>

Byte-Pair Encoding (BPE): training

- Build the vocabulary through successive applications of “merge” operations
 - training corpus: **low** (5 times), **lower** (2 times), **newest** (6 times), **widest** (3 times)
 - the initial vocabulary, after pre-processing: `l o w </w> e r n s t i d`
- **1st step**: what is the most frequent character bigram?
 - **e+s** occurs 9 times
 - add new symbol '**es**' (merged from **e+s**) to vocabulary
 - resulting vocabulary: `l o w </w> e r n s t i d es`
- **2nd step**: what is the most frequent bi-gram of symbols?
 - **es+t** occurs 9 times (note that 'es' now participates in the counts)
 - add new symbol '**est**' to vocabulary (still keeping 'e', 's' and 'es')
 - resulting vocabulary: `l o w </w> e r n s t i d es est`
- After 5 steps, vocabulary is: `l o w </w> e r n s t i d es est est</w> lo low`

Byte-Pair Encoding (BPE): results & testing

- Result of training consists of the **vocabulary** and a set of “merge” rules
 - vocabulary contains
 - **all characters** ($10^2 - 10^4$), **frequent words** (often >60% of all items), and **frequent character n-grams**: some are meaningful (e.g., prefixes or suffixes) but many are not
 - vocabulary size = number of initial characters + number of merges – X
 - often between 10k and 32k, more for CJK or large models (up to 256k, [Tao et al. 2024](#))
 - X : some subwords are removed if they occur only as parts of larger subwords
- **Testing mode: tokenize new text**
 - preprocess, including splitting into characters
 - apply the merge operations learned, in the same order
 - no new counts are needed
 - **example (unseen)**: `lowest</w>` → `low`  `est</w>`

<code>e s</code>	→	<code>es</code>
<code>es t</code>	→	<code>est</code>
<code>est </w></code>	→	<code>est</w></code>
<code>l o</code>	→	<code>lo</code>
<code>lo w</code>	→	<code>low</code>

WordPiece: training

- WordPiece builds the vocabulary similarly to BPE, through successive applications of ‘merge’ operations starting from a character-based vocabulary
 - start with a vocabulary of individual characters from the training corpus
 - add [UNK] for unknown tokens and the ## prefix for subwords that don’t start a word
 - split training corpus into words and create all possible subword combinations for each word
 - count all subword frequencies
 - evaluate possible new subwords, i.e. candidate merges $x\ y$ using a scoring function:

$$P(xy) / (P(x) P(y))$$
 rationale = proxy for likelihood improvement
 - add to vocabulary the new subword with the highest score (possibly with ## prefix)
 - update frequencies of affected tokens, then redo ‘evaluate ...’
 - stop when desired vocabulary size is reached (or highest score below threshold)

WordPiece: results & testing

- Result of training consists of the vocabulary
 - vocabulary
 - all characters, frequent words, subwords (e.g. 'quickness' = 'qu' + '##ick' + '##ness'), and the UNK symbol for completely new words
 - size between 30k (Devlin et al. (2018)) and 110k for multilingual BERT (Pires et al. (2018))
- Testing mode: tokenize new text
 - preprocess, including splitting into words
 - for each word, match the longest subword from the vocabulary at the start of the word
 - if a match is found, add it to the output and repeat from the next character, for matches after the first character, use the ## version of the subword
 - if no match is found, add [UNK] token and move to the next character

Comparison of subword tokenization methods

	BPE	WordPiece	UnigramLM
Training	Starts from a <i>small</i> vocabulary and learns rules to <i>merge</i> tokens	Starts from a <i>small</i> vocabulary and learns rules to <i>merge</i> tokens	Starts from a <i>large</i> vocabulary and learns rules to <i>remove</i> tokens
Training steps	Merges the tokens corresponding to the most <i>frequent</i> pair	Merges the tokens corresponding to the pair with <i>the best score</i> (frequency of the pair, privileging pairs where each individual token is less frequent)	Remove <i>n%</i> tokens corresponding to the lowest reduction in likelihood upon removal computed on the whole corpus
Result of training	“Merge” rules and a vocabulary	Just a vocabulary	Vocabulary and score for each token
Encoding a new text	Splits words into characters and applies the merges learned during training	Finds the longest subword (starting from the beginning) that is in the vocabulary, then does the same for the remainder of the word	Finds the most likely split into tokens, using the scores learned during training

SentencePiece framework

- SentencePiece directly operates on **raw text without pre-tokenization/pre-splitting**,
 - text is considered as a sequence of Unicode characters
 - whitespace is replaced with the special character "▬" (U+2581)
 - text is normalized via Unicode NFKC normalization (default) or custom normalization rules
- Token vocabulary of predefined size is created by **BPE** or **UnigramLM** on raw text
- Advantages
 - does not require language-specific pre-tokenization
 - is readily applicable for languages w/o whitespace-separators (e.g., Chinese or Japanese)
 - improves NMT accuracy and robustness via **subword regularization** using multiple subword segmentations, e.g., "internationalization" → [international, alization] or [international, ization]

SentencePiece framework: results & testing

- Result of training is a fully self-contained model with:
 - pre-compiled finite state transducer for character normalization
 - vocabulary and segmentation parameters with:
 - all characters, many frequent words, subwords, [UNK] if completely new words
 - vocabulary size
 - is around 32k for T5 (cf. [Raffel et al. \(2019\)](#)) & Llama (cf. [Touvron et al. \(2023\)](#))
- Testing mode: tokenize new text
 - use SentencePiece for offline tokenization, or integrate it into a pipeline
 - segmentation speed of SentencePiece is high (around 10^4 — 10^5 sentences/sec)

Kudo. T., & Richardson, J., “[SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing](#)”, 2018

Kudo, T.. “[Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates](#)”, 2018

C. Tao et al., “[Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies](#)”, Arxiv:2407.13623, 2024.

Conclusion

- Subword tokenization
 - splits text into subword units and reduces vocabulary size compared to word-based tokenization
 - effectively handles out-of-vocabulary words and captures certain semantic relationships between words and their variations
 - is more complex to implement & produces longer sequences than word-based tokenization

```
] 1 tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-cased')
```

```
] 1 print(tokenizer.tokenize('There are eight orbiters surveying  
2 the planet: Mars Odyssey, Mars Express, Mars Reconnaissance  
3 Orbiter, Mars Orbiter Mission, MAVEN, the Trace Gas Orbiter,  
4 the Tianwen-1 orbiter, and the Hope Mars Mission'))
```

```
['There', 'are', 'eight', 'orbit', '##ers', 'surveying', 'the', 'planet', ':', 'Mars', 'Odyss  
ey', ',', 'Mars', 'Express', ',', 'Mars', 'Reconnaissance', 'Or', '##bit', '##er', ',', 'Mars',  
, 'Or', '##bit', '##er', 'Mission', ',', 'MA', '##VE', '##N', ',', 'the', 'Trace', 'Gas', 'O  
r', '##bit', '##er', ',', 'the', 'T', '##ian', '##wen', '-', '1', 'orbit', '##er', ',', 'and',  
, 'the', 'Hope', 'Mars', 'Mission']
```

3. CONDITIONAL TEXT GENERATION & STATISTICAL LANGUAGE MODELS

Language models (LMs): Definition & application

- Definition: given a word sequence, an LM predicts the **most likely next word** or serves to compute the **probability of any sequence of words**
- LMs are subcomponent of many NLP tasks
 - spelling and grammar correction $P(\text{"my cat eats fish"}) > P(\text{"my cat eats dish"})$
 - speech recognition $P(\text{"recognize speech"}) > P(\text{"wreck a nice beach"})$
 - machine translation $P(\text{"the red house"}) > P(\text{"the house red"})$
 - predictive typing (autocompletion)
 - OCR and handwriting recognition
 - authorship identification
 - summarization
 - chatbots (e.g., ChatGPT)
 - information retrieval

Formal definition

- LMs are models/tools that
 - either compute the joint probability of a sequence of words $P(w_1, w_2, w_3, \dots, w_n)$
 - or compute the conditional probability of a next word $P(w_n \mid w_1, w_2, w_3, \dots, w_{n-1})$
 where $P(w_i) = P(W = w_i)$ denotes the probability of word w_i to occur

- The two formulations are equivalent because

$$P(w_n \mid w_1, w_2, w_3, \dots, w_{n-1}) = \frac{P(w_1, w_2, w_3, \dots, w_n)}{P(w_1, w_2, w_3, \dots, w_{n-1})}$$

- Therefore, going from left to right (earlier to later words) via chain rule:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \cdot \dots \cdot P(w_n \mid w_1, \dots, w_{n-1}) = \prod_{k=1}^n P(w_k \mid w_1, \dots, w_{k-1})$$

Evaluating language models

- **Extrinsic evaluation**: how much does a LM help a task?
 - Complex, computationally expensive, task-specific assessment
- **Intrinsic evaluation: perplexity**
(train LM, evaluate LM on unseen test set)
 - probability $P(w_1, w_2, w_3, \dots, w_N) \rightarrow$ the higher, the better
 - perplexity $PP(w_1, w_2, w_3, \dots, w_N) \rightarrow$ the lower, the better
- Example¹:
 - Models: Unigram, Bigram and Trigram
 - Train set: $38 \cdot 10^6$ words from Wall Street Journal (vocab: 20k words)
 - Test set: 1.5 million words
 - Result: $\rightarrow$ higher n-grams $\rightarrow$ more info about word sequence
 $\rightarrow$ lower perplexity

LM perplexity always to be confirmed by extrinsic evaluation of LM

$$\begin{aligned}
 PP(W) &= P(w_1, w_2, w_3, \dots, w_N)^{-\frac{1}{N}} \\
 &= \sqrt[N]{\frac{1}{P(w_1, w_2, w_3, \dots, w_N)}} \\
 &= \sqrt[N]{\prod_{k=1}^N \frac{1}{P(w_k | w_1, w_2, w_3, \dots, w_{k-1})}} \\
 &= 2^{H(W)} \\
 &= 2^{-\frac{1}{N} \log_2(P(w_1, w_2, w_3, \dots, w_N))}
 \end{aligned}$$

	Unigram	Bigram	Trigram
$PP(W)$	962	170	109

¹ Source: Jurafsky, D. & Martin, J.H. (2008), "Speech and Language Processing" (3rd edition, <https://web.stanford.edu/~jurafsky/slp3/>, Chapter 3.2) 23

Parenthesis: Machine Translation (MT)

- Goal: Automatically translate a sentence or document from a source language to a target language, preserving the meaning, tone, and fluency
- Application domains include real-time communication, medical report or legal documents translation or multilingual customer/service support
- Sentence complexity, idioms, and domain-specific vocabulary affect translation quality, target language pairs is a parameter of the task determining difficulty of task
- Methodologically there are 2 main approaches:
 1. **Rule-based / Statistical Translation** (before 2015), "assembled" translations from pieces of text using rules or aligned phrases
 2. **Neural Machine Translation** (after 2015), translates by generating new text in the target language, like a human translator would; approach involves training on source-target parallel corpora and is enabled by the fast improvements of LMs conditioned on source text; **seq2seq**

Parenthesis: Automatic summarization

- Goal: create a summary of a given text or several texts preserving core meaning and key information
- Application domains include news aggregation, scientific article digests, legal or medical report summarization, chat and meeting summaries
- Text type, e.g., structure, tone, and density of the input text matter, summary length is a parameter of the task
- Methodologically there are 2 main approaches:
 1. **Extractive**, selects most representative sentences of the text without reformulation; coherence of abstract is problematic, but often appropriate for news (hint: take first sentences, faithfulness)
 2. **Abstractive**, generates new, shorter text by rephrasing and compression, just as humans do; approach involves training on text-summary pairs and is enabled by the fast improvements of LMs conditioned on source text; **seq2seq**

Commonality: conditional text generation

- We look for **language models**: $P(w_n \mid C, w_1, w_2, w_3, \dots, w_{n-1})$ where C is a “context”
 - pure language model: $C = \emptyset$
 - machine translation: C = source language text
 - summarization: C = original article
 - other options, e.g., question answering or paraphrasing or image captioning
- Models thus **predict the next token**: $\operatorname{argmax}_{w_n \in V} P(w_n \mid C, w_1, w_2, w_3, \dots, w_{n-1})$
 - applied several times in order to generate a larger text = “decoding”
- If C is a sequence of tokens: **sequence-to-sequence** model (seq2seq)

n -gram LM and Markov assumption

- Simplest LM is based on sequences of words, the n -gram
- Markov chain assumption: $P(w_n | w_1, \dots, w_{n-1}) \approx P(w_n | w_{n-m}, \dots, w_{n-1})$
- The order of LM is defined by m often:

$$P(w_1, w_2, \dots, w_n) = \prod_{k=1}^n P(w_k | w_1, \dots, w_{k-1})$$

– $m = 0$ (unigrams) $\longrightarrow$

$$P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k)$$

– $m = 1$ (bigrams) $\longrightarrow$

$$P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

– $m = 2$ (trigrams) $\longrightarrow$

$$P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-2}, w_{k-1})$$

- Example: with trigrams

?

?

$$P(\text{the}, \text{students}, \text{opened}, \text{their}) \approx P(\text{the} | \langle s \rangle, \langle s \rangle) \cdot P(\text{students} | \langle s \rangle, \text{the}) \cdot P(\text{opened} | \text{the}, \text{students}) \cdot P(\text{their} | \text{students}, \text{opened})$$

Learning a language model (LM)

- To compute the probability of a sequence, find the probabilities of its n -grams
 - Example with **trigrams**: what is our best estimate of $P(w_k | w_{k-2}, w_{k-1})$?
- Use a large training corpus and apply **maximum likelihood estimation (MLE)** based on the idea that “what you see is in fact the most likely event”
 - Use **relative frequencies**, i.e., **count** numbers of occurrences of **trigrams** and **normalize** counts so that they lie between 0 and 1

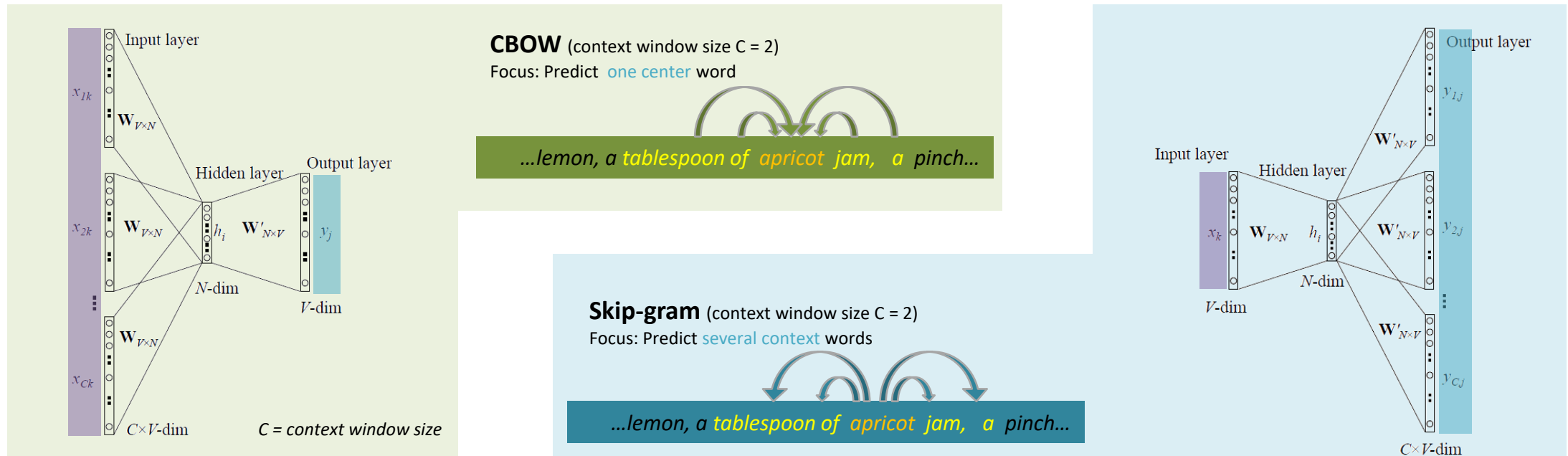
$$P(w_k | w_{k-2}, w_{k-1}) = \frac{\text{count}(w_{k-2} w_{k-1} w_k)}{\sum_{w \in \text{Vocab}} \text{count}(w_{k-2} w_{k-1} w)} = \frac{\text{count}(w_{k-2} w_{k-1} w_k)}{\text{count}(w_{k-2} w_{k-1})}$$

- The **larger** the training **corpus**, the **better** the **estimations...** but the **more** n -grams must be **stored** in the model... and **increasing** n also **increases model size!**

4. NON-CONTEXTUAL WORD EMBEDDINGS WITH NEURAL NETWORKS

Computing word embeddings with neural nets

- Training a **neural network** language model, was observed to also yield **learned word representations**, which can be used for other, potentially unrelated, tasks
- Thus, **novel model architectures** for **efficiently computing** continuous vector representations of words from – at that time – very large data sets were developed

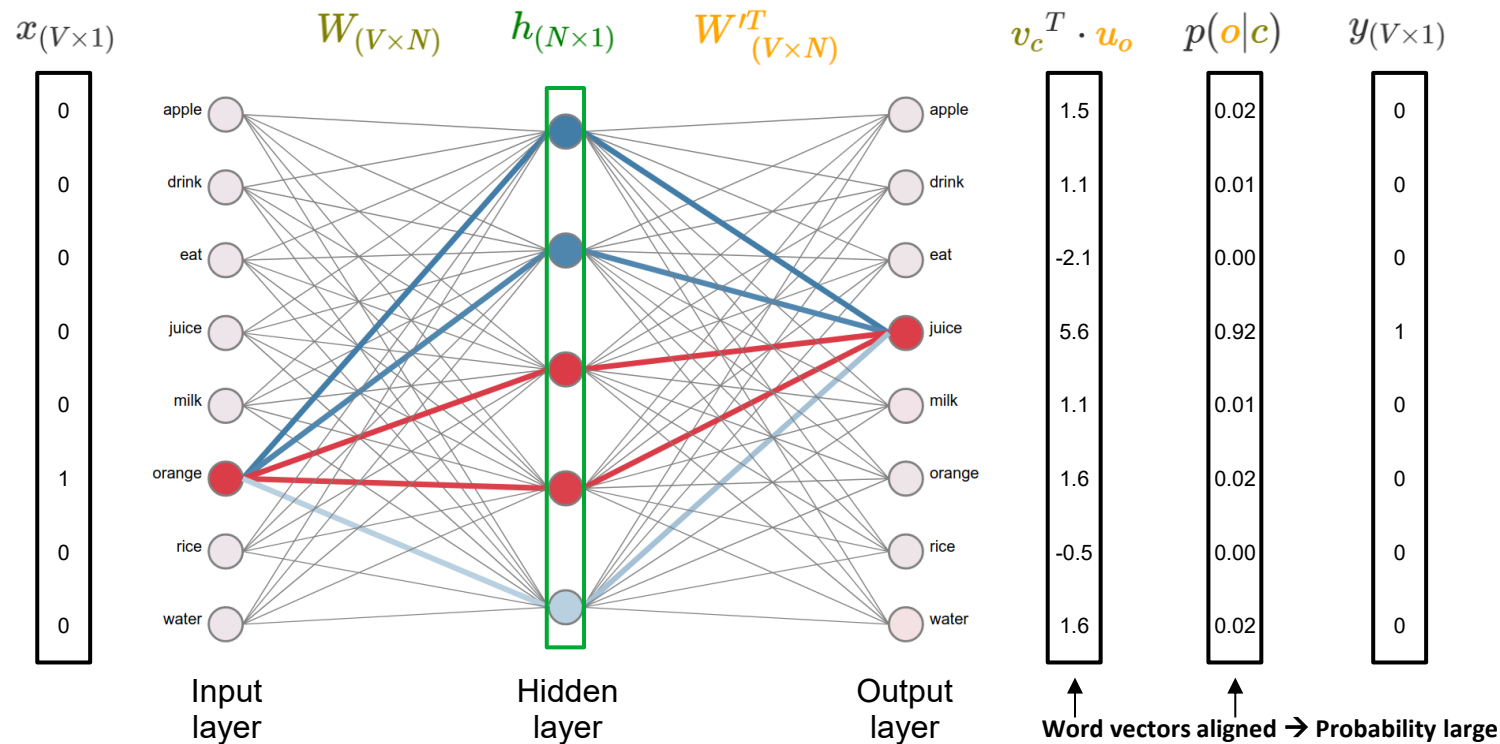


Computing word embeddings with word2vec

- The **superficial classification task** of predicting the most likely upcoming words is used to resolve the **underlying objective**, i.e., to **learn dense word embeddings** that reflect semantic relationships
- The architecture used is a **multi-layer perceptron** with **three layers** (input, hidden and output layer), the hidden layer uses a linear activation function
- The **dimensionality of word embeddings** is defined by the size of the hidden layer and typically between 200 and 500
- **Dense word embeddings** are extracted from connections between input and hidden layer (connections between hidden layer and output layer are context embeddings)
- **word2vec** trains the neural network (much) **more efficiently** than Truncated SVD

The word2vec network

1-hot vector (center word) weight matrix W (input-hidden) weight matrix W' (hidden-output) inner product softmax 1-hot vector (context word)



Notes

- the **probability calculation** is based on embedding similarity of **center** v_c and **context** word u_o expressed as **inner product** $v_c^T \cdot u_o$

Note

$$v_{w_i} = W^T \cdot x_{w_i} = h_{w_i}$$

$$W'^T \cdot h_{w_i} = u_{w_i}$$

- The dot product "similarity" is **translated into probabilities** by applying the **softmax function** over all output words in the output layer

$$p(o|c) = \frac{e^{v_c^T \cdot u_o}}{\sum_{w \in V} e^{v_c^T \cdot u_w}}$$

Computing word embeddings with word2vec

- Both word2vec models, i.e., CBOW and Skip-Gram learn two embeddings for each word via W and W' . Word “i” is represented by the i th row of (a) W or (b) $W + W'$
- CBOW (Continuous Bag-of-words) predicts one center word
 - “Continuous” refers to the non-discrete representation used by the model
 - Context words on the input side are embedded into the hidden layer
 - Embedded context words are averaged before projecting them to the output layer
- SG (Skip-gram) predicts several context words
 - Word pairs in training are not all proper bi-grams, but often involve “skipping over” some words
 - Each context word on the output is treated equally in probability computations, the model only has one output layer
 - Skip-gram takes much longer to train! It produces many more training samples than CBOW

Training the Skip-gram word2vec model

- The Skip-gram model maximizes the probability with the context set of w , $C(w)$

$$\operatorname{argmax}_{\Theta} \prod_{w \in \text{Text}} \left[\prod_{c \in C(w)} p(c|w, \theta) \right]$$

- Theoretical solution is found via gradient descent
 - start from random matrices, consider difference between desired and actual outputs on the data
 - compute the gradient for each parameter (weight) and back-propagate it to lower layers
 - adjust coefficients of W' , then W , s.t. difference desired vs. actual predictions is reduced
- The basic Skip-gram formulation defines $p(c|w, \theta)$ using the **softmax function**, which is **computationally expensive**, due to **summation in the denominator**

Training the Skip-gram word2vec model

- The solution described above has further **limitations**
 - it is impossible to perform gradient descent for all data at once (approximation: do it for each data point individually)
 - training converges towards a degenerate solution, i.e., $w = c$ and $w^T c = K$, where $K \approx 40$ or larger
- This limitation was solved by **adding for each positive sample, several negative samples**, and only update a small fraction of model weights $\rightarrow$ SGNS
 - draw J random words w from the data, suppress influence of frequent words via $\text{freq}(w)^{3/4}$
 - perform gradient descent over positive and negative samples

Skip gram negative sampling (SGNS)		
Sentence ²⁾ (context window of size 2)		
...lemon, a [tablespoon of apricot jam, a] pinch...		
context 1 c2 target c3 c4		
Samples	Positive	target context
		apricot tablespoon
		apricot of
		apricot jam
		apricot a
	Negative	apricot aardvark
		apricot my
		apricot where
		apricot coaxial
		apricot seven
		apricot forever
		apricot dear
		apricot if
		k = 2, i.e. more neg. than pos. samples

Skip-gram model with negative sampling (SGNS)

- Given positive and negative instances and initial embedding vectors, the training goal is to adjust word vectors with MLE
 - maximize similarity drawn from the **positive data** for the target word, context word pairs w, c_{pos}
 - minimize similarity drawn from the **negative data** for the target word, context word pairs w, c_{neg}
- In other words, **maximize dot product** of the word with the **actual context word** and **minimize dot products** of the word with the **k negative sampled non-neighbor words**

$$\begin{aligned}
 L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\
 &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]
 \end{aligned}$$

$$P(+|w, c)$$

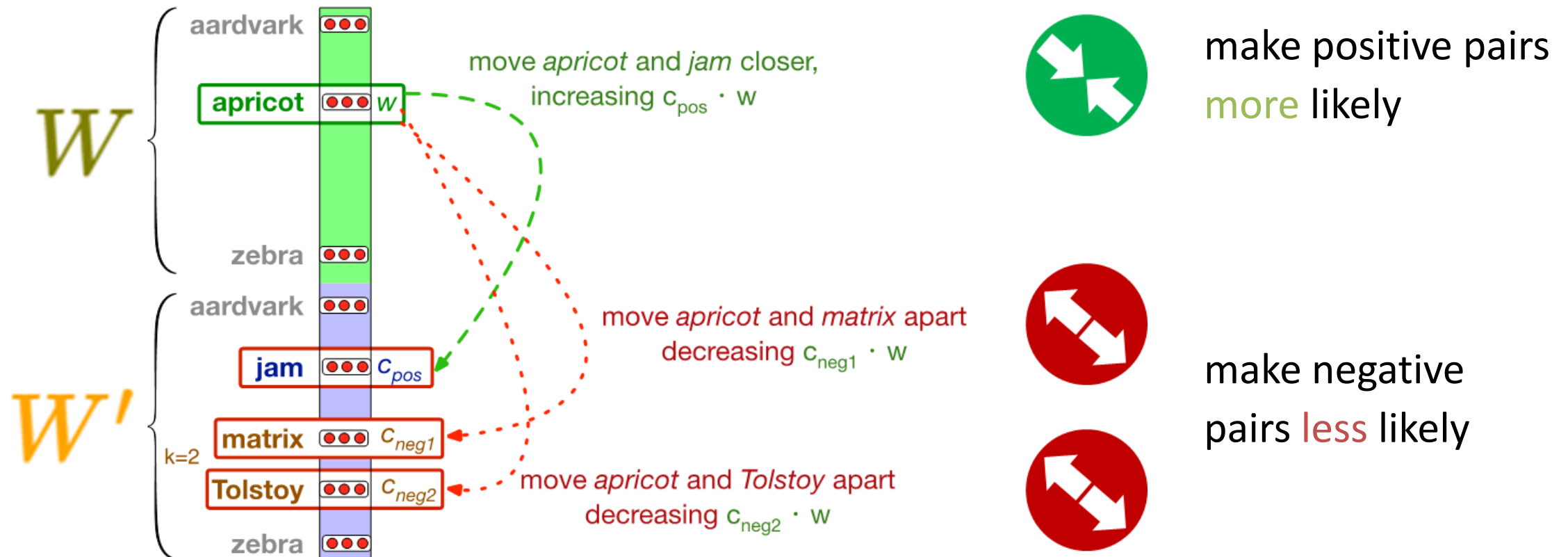
... probability that c is a context word

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

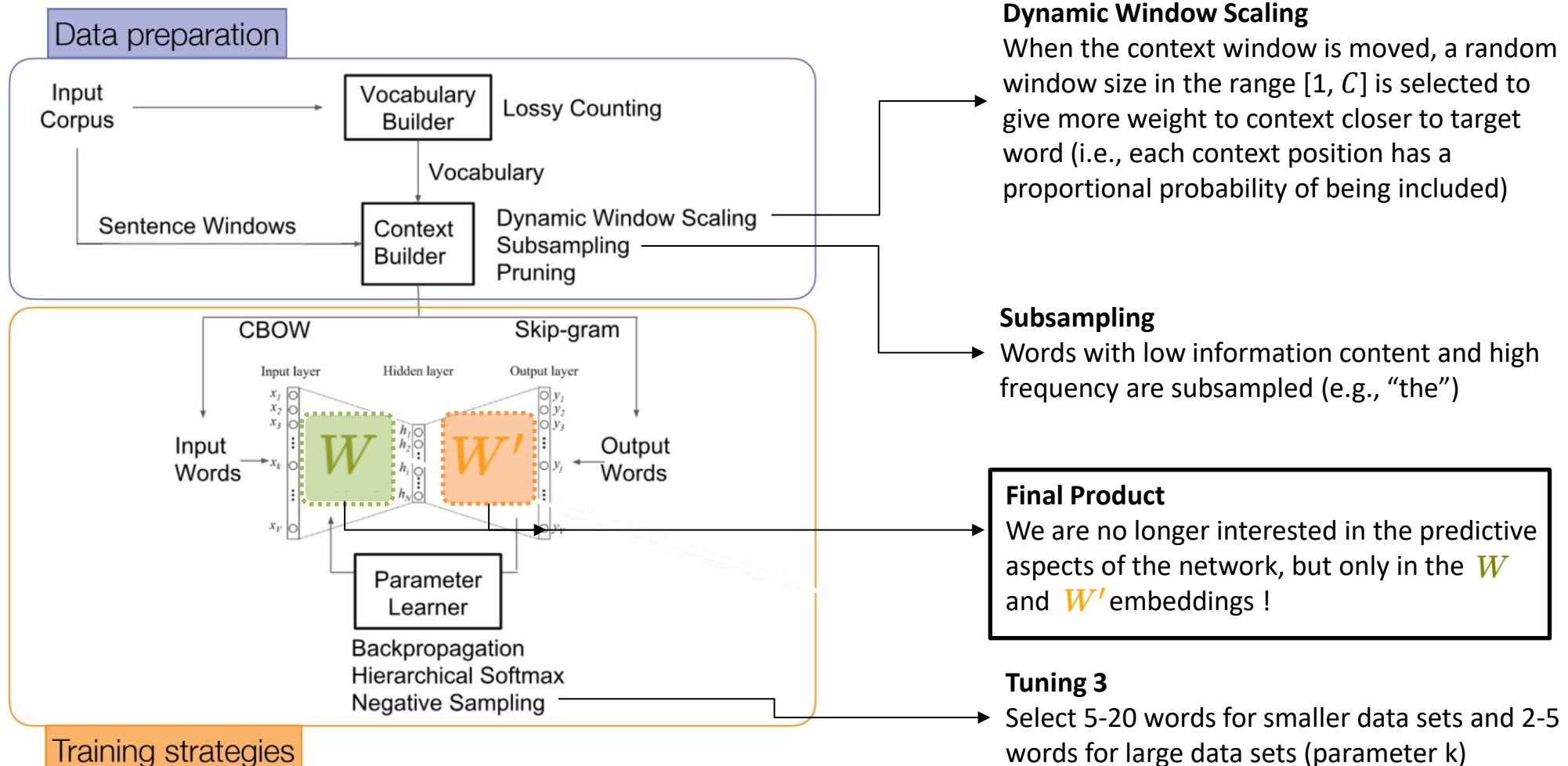
... transforms similarity to probability

SGNS - One step of stochastic gradient descent

Positive word pair “**apricot** **jam**”

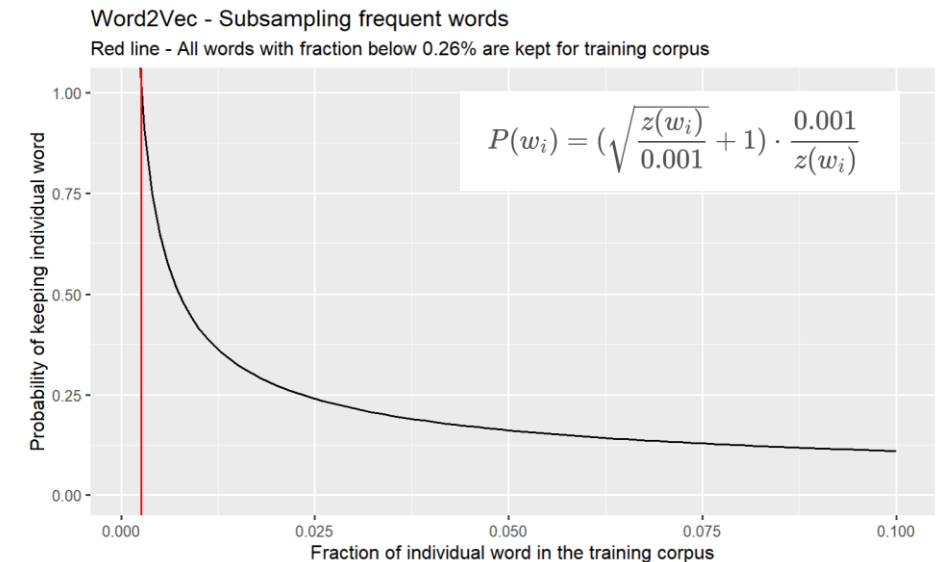
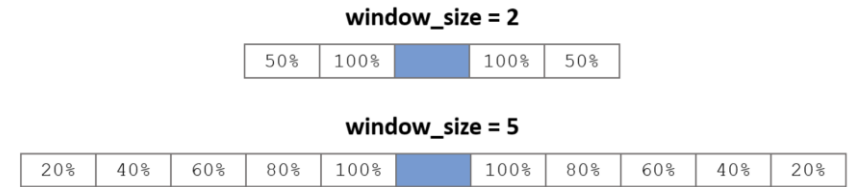


Word2vec – Design & tuning considerations



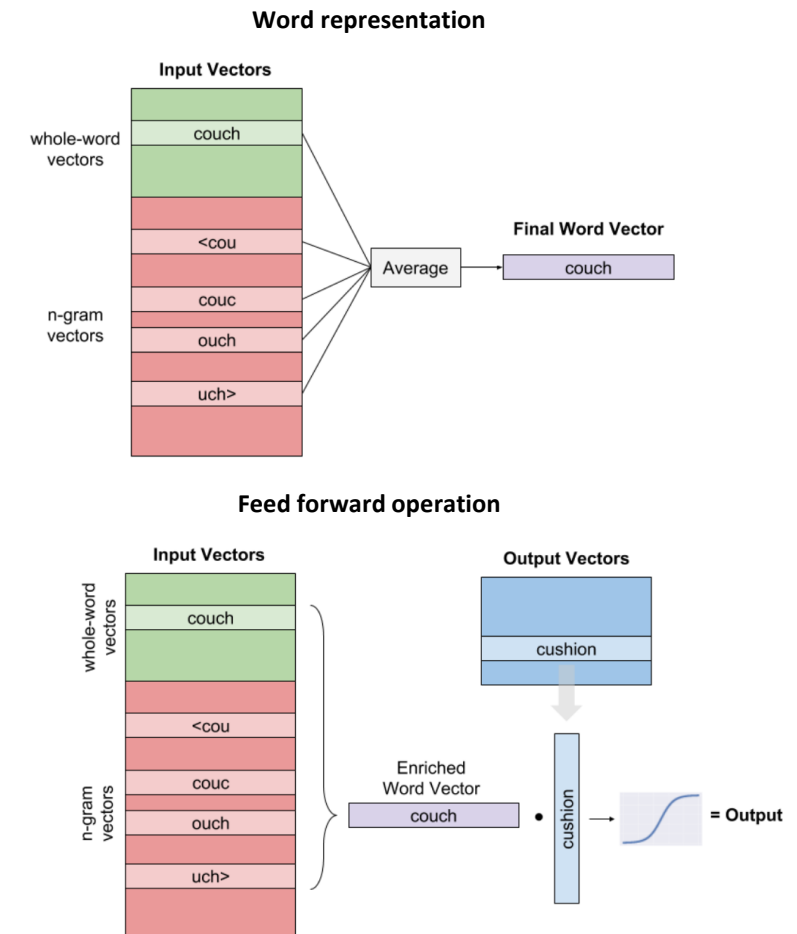
Word2vec – Design & tuning considerations

- Dynamic window scaling
 - Given a maximum context window of size C , a random window size in the range $[1, C]$ is selected, each time the context window is moved to the next word in a sentence → more weight given to context closer to target word
- Subsampling frequent and irrelevant words
 - Frequent words are only kept in the training vocabulary with a certain probability; words which represent less than 0.26% of the total words will never be subsampled
 - Note: Do not mix up subsampling of the vocabulary with negative sampling adjustment based on $freq(w)^{3/4}$



fastText – Solution for handling rare words

- fastText extends word2vec to learn from less data and represent rare words and typos, w/o major architectural changes
- fastText enhances vocabulary by adding a collection of character n -grams of which a word is composed; by default, n takes on values 3, ..., 6
- At inference, rare words not present in the training data are represented by the average of underlying n -grams
- fastText needs to learn substantially more token vectors, hence more time and memory... and vectors for common words are not necessarily better



Conclusion

- Learnt word embeddings revolutionized NLP with various applications, e.g., information retrieval, similarity analysis, word clustering, sentiment analysis, and topic classification
- Neural prediction models (e.g., word2vec & fastText) differ from other methods which use co-occurrence statistics (e.g., LSA & GloVe)
- All of these methods yield static or non-contextual word embeddings, i.e., a single vector for ambiguous words like 'bank', 'bat' or 'apple'
- At the text-level (sentence, document), the embeddings can be computed by averaging and normalizing word vectors
- The next big improvement was to build sentence embeddings considering the order of words → Transformers

APPENDIX

Self study – Evaluating LMs with *BLEU* for MT

- The *BLEU* metric ranks each MT target sentence by function of the number of n -gram overlaps with the human translation and is based on precision

Source	la verdad, cuya madre es la historia, émula del tiempo, depósito de las acciones, testigo de lo pasado, ejemplo y aviso de lo presente, advertencia de lo por venir.
Reference	truth, whose mother is history, rival of time, storehouse of deeds, witness for the past, example and counsel for the present, and warning for the future.
Candidate 1	truth, whose mother is history, voice of time, deposit of actions, witness for the past, example and warning for the present, and warning for the future
Candidate 2	the truth, which mother is the history, émula of the time, deposition of the shares, witness of the past, example and notice of the present, warning of it for coming

Candidate 1 shares **many more n-grams** (cf. boxes) and especially **longer n-grams** (cf. darker boxes) with the reference translations than Candidate 2

Self study – Evaluating LMs with *BLEU* for MT

- *BLEU* (Papineni et al., 2002) is evaluated on a corpus and ranges from 0 to 1, built by comparing reference translation r and candidate text c
 1. compute n -gram matches sentence by sentence
 2. clip n -gram counts for all the candidate sentences
 3. divide by number of candidate n -grams in the corpus
 4. compute geometric mean across all n -grams
 5. multiply by brevity penalty BP
- Practical considerations: 2-4 ref. translations (concatenated), often $N = 4$, $w_n = 1/N$

$$\begin{aligned}
 BLEU &= BP \cdot \exp \left(\sum_{n=1}^N w_n \cdot \log(p_n) \right) \\
 &= \min \left(1, e^{1-\frac{r}{c}} \right) \cdot \prod_{n=1}^N prec_n^{w_n} \\
 prec_n &= \frac{\sum_{Cand} \sum_{n=1}^N count_{match,clipped}(n-gram)}{\sum_{Cand} \sum_{n=1}^N count(n-gram)} \\
 r / c &: \# \text{ words in reference} / \text{ candidate translation}
 \end{aligned}$$

BLEU is quick to calculate, easy to understand, language-independent, but considers only exact word matches w/o semantic meaning or word importance

Self study – Evaluating LMs with *BLEU* - examples

- Reference: “The guard arrived late because it was raining”

- Candidate: “The guard arrived late because of the rain”

n	1	2	3	4
n-gram precision	5/8	4/7	3/6	2/5

- n-gram precision = # n-gram matches / # total candidate n-grams

- Geometric mean = $\exp\left(\sum_{n=1}^4 w_n \cdot \log(p_n)\right) = (5/8)^{\frac{1}{4}} \cdot (4/7)^{\frac{1}{4}} \cdot (3/6)^{\frac{1}{4}} \cdot (2/5)^{\frac{1}{4}}$

- Brevity penalty = $\min(1, e^{1-\frac{r}{c}}) = \min\left(1, e^{1-\frac{8}{8}}\right) = 1$

n-grams = 1...N
weights = 1/N

- Reference: “The cat is on the mat”

- Candidate: “The the cat is is on the mat”

Candidate 1-gram “the” matches and occurs 3 times,
but only occurs 2 times in reference, count clipped to 2!

- Modified n-gram precision = $prec_n = \frac{\sum_{Cand} \sum_{n=1}^N count_{match,clipped}(n-gram)}{\sum_{Cand} \sum_{n=1}^N count(n-gram)}$

Self study – Evaluating LMs with *ROUGE* for summarization

- *ROUGE* is evaluated between a candidate and a set of reference texts via matching n -grams and ranges from 0 to 1

1. compute N -gram matches between candidate c & all references r_i
2. add up number of matches for all reference sentences r_i
3. divide by the sum of reference N -grams

$$ROUGE - N = \frac{\sum_{S \in Refs} \sum_{N-gram \in S} count_{match}(N - gram)}{\sum_{S \in Refs} \sum_{N-gram \in S} count(N - gram)}$$

Note:

- *ROUGE-N* defines N (e.g., 4) and only considers N -grams (unlike *BLEU*),
- inner sum over N -grams in a given sentence, outer sum over all reference sentences

ROUGE is an acronym for Recall-Oriented Understudy for Gisting Evaluation

- NB: *ROUGE-1* focusses on keywords, *ROUGE-4* on syntactic & linguistic structure, *ROUGE-N_{multi refs}* considers max. pairwise summary-level *ROUGE-N*, *ROUGE-L* the longest common subsequence
- *ROUGE* (C. Lin, 2004) is quick to calculate, easy to understand, language-independent, but considers only exact word matches w/o semantic meaning or word importance

Self study – Evaluating LMs for summarization

- Other n -gram based metrics are also used, e.g., *BLEU* or METEOR
 - METEOR computes the harmonic mean of unigram precision and recall and incorporates stemming and synonym matching for better alignment with human judgments.
- Embeddings help to overcome some of the limitations of n -gram based metrics
 - BERTScore computes cosine similarity between contextual embeddings, capturing semantic meaning beyond exact word matches.
 - BLEURT fine-tunes BERT on human evaluation data, providing robust, trained assessment of summarization quality.
 - Value-Add: recognizing paraphrasing, reordering & meaning preservation with different wording
- Human evaluation remains important for several reasons
 - Grammaticality, referential clarity, conciseness, fluency, coherence, coverage, faithfulness, etc.

Self study – Applying a language model

- Compute the probability of the sentence "*I want to eat Chinese food*" using a **bigram model**, i.e., $P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Count → Probs.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram counts and probabilities for eight words in the Berkeley Restaurant Project corpus with unigram counts tabulated in blue (vocabulary size = 1446, number of sentences = 9332, $P(i | <s>) = 0.25$, $P(</s> | food) = 0.68$).

$$\begin{aligned}
 P(< s > I \text{ want to eat Chinese food } < \backslash s >) &= P(I | < s >) \cdot P(\text{want} | I) \cdot P(\text{to} | \text{want}) \cdot P(\text{eat} | \text{to}) \cdot P(\text{Chinese} | \text{eat}) \cdot P(\text{food} | \text{Chinese}) \cdot P(< \backslash s > | \text{food}) \\
 &= 0.25 \times 0.33 \times 0.66 \times 0.28 \times 0.021 \times 0.52 \times 0.68 = 0.00011
 \end{aligned}$$

Self study – Limitations: unseen n -grams

- Approximating n -gram probabilities with counts works well for frequent n -grams,

$$P(\textit{nextword} | \textit{students}, \textit{opened}, \textit{their}) = \frac{\text{count}(\textit{students opened their nextword})}{\text{count}(\textit{students opened their})} \quad \text{but...}$$

- numerator:** if “*students opened their nextword*” never occurred in the training corpus, then conditional probability equals zero!
 - denominator:** if “*students opened their*” never occurred in the training corpus, then we can’t even calculate probability for a next word at all!
- Certain n -grams might **not be present** in the **training corpus** but **appear** during **inference** → For **new sentence containing them**, we get **zero probability!**
 - Increasing n , i.e., using **higher order n -grams increases** these two **sparsity problems!**

Self study – Limitations: unseen n -grams

- Solution to sparsity problem #1: **Smoothing**
 - shave off a bit of probability mass from some more frequent n -grams and give it to the n -grams that were never seen
 - add a **fictitious count** (a small number) to **each** probability, to avoid unseen n -grams having 0 probability
- Example for tri-grams:

Other smoothing methods

1. Deleted estimation
 2. Good-Turning
 3. Witten-Bell
 4. Kneser-Ney
- Or backoff & interpolation

$$P(w_k | w_{k-2}, w_{k-1}) = \frac{\text{count}(w_{k-2} w_{k-1} w_k)}{\sum_{w \in V} \text{count}(w_{k-2} w_{k-1} w)} \approx \frac{\text{count}(w_{k-2} w_{k-1} w_k) + 1}{(\sum_{w \in V} \text{count}(w_{k-2} w_{k-1} w)) + N_{\text{trigram}}}$$

improved version...

$$\approx \frac{\text{count}(w_{k-2} w_{k-1} w_k) + k \cdot 1}{(\sum_{w \in V} \text{count}(w_{k-2} w_{k-1} w)) + k \cdot N_{\text{trigram}}}$$

Laplace smoothing
with normalization

Reduce correction and estimate n with held-out data
(compare with actual counts of unseen n -grams)

Self study – Other solutions for unseen n -grams

- Solution to sparsity problem #2: **backoff & interpolation**
 - if we have no examples of a particular trigram, **using less context** helps to generalize
 - **Backoff**: if a count is zero for an n -gram, then **use lower-order n -grams**
 - **Interpolation**: **combine lower-order n -grams**, e.g., through **linear interpolation**
 - compute counts on training data for all lower-orders, combine language models of different orders with positive weights, λ_i , $\sum_i \lambda_i = 1$, **constant or context-specific**, tune weights on an additional held-out corpus

$$P(w_k | w_{k-2}, w_{k-1}) \approx \lambda_1 \cdot P(w_k) + \lambda_2 \cdot P(w_k | w_{k-1}) + \lambda_3 \cdot P(w_k | w_{k-2}, w_{k-1})$$

- Desired effect: a zero count for an unseen trigram does not lead to a zero interpolated probability